

Documentarea folosirii tool-urilor AI

Here's exactly how you can do it:

1. Create a Custom Validation Attribute

Create a new class like this:

```
csharp Copy Edit

using System.ComponentModel.DataAnnotations;

public class GitHubUrlAttribute : ValidationAttribute
{
    protected override ValidationResult IsValid(object? value, ValidationContext validationContext)
    {
        if (value is string url && !string.IsNullOrEmpty(url))
        {
            if (!url.Contains("github.com"))
            {
                return new ValidationResult("Trebuie sa fie un link catre GitHub (ex: https://github.com/...");
            }
        }
        return ValidationResult.Success!;
    }
}
```

Similarly, you can create a `LinkedInUrlAttribute`:



To pull without losing your work when the remote branch has unrelated changes, **you should use** `git pull --rebase`. This approach keeps your local commits on top of the updated remote history, avoiding unnecessary merge commits.

Here's how to do it safely:

✓ Recommended Steps

1. Stash any uncommitted changes (optional but safe):

```
bash Copy Edit

git stash
```

2. Pull with rebase:

```
bash Copy Edit

git pull --rebase origin develop
```

This does the following:

- Fetches the remote changes.
- Reapplies your local commits on top of the updated branch.

3. Resolve any merge conflicts if prompted. Git will guide you through resolving them.

4. After resolving conflicts (if any), continue the rebase:

```
bash Copy Edit

git rebase --continue
```

write me a python script that processes get requests

Sure! Here's a simple Python script that uses the **Flask** framework to process GET requests. It runs a web server and responds to GET requests at different routes.

✅ Requirements:

- Python installed
- Flask (`pip install flask`)

✅ Example: `app.py`

```
python                                                                    Copiază Editează

from flask import Flask, request, jsonify

app = Flask(__name__)

# Root endpoint
@app.route("/", methods=["GET"])
def home():
    return "Welcome to the GET request processor!"
```

i want to use requests package not flask

Got it! Here's a simple Python script using the `requests` package to **send** GET requests and **process the responses**.

The `requests` package is used for **sending HTTP requests**, not for handling them like Flask. So this script will **send a GET request to a server**, get the response, and process it.

✅ Example: Using `requests` to send GET requests

```
python                                                                    Copiază Editează

import requests

def get_data_from_api(url, params=None):
    try:
        response = requests.get(url, params=params)
        response.raise_for_status() # Raises an error for bad status codes

        print("Status Code:", response.status_code)
        print("URL:", response.url)
        print("Headers:", response.headers)
        print("Response JSON:", response.json()) # If response is JSON
        return response.json()
```

no. i want to respond to get requests

Thanks for clarifying! If you want to **respond to GET requests** *without* using Flask, then you'll need to create a simple HTTP server manually.

Here's how you can do it using Python's built-in `http.server` module, which can handle GET requests:

✓ Example: Basic GET Request Handler (No Flask)

python

📄 Copiază

✎ Editează

```
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlparse, parse_qs
import json

class SimpleGetHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        # Parse the query string
        parsed_path = urlparse(self.path)
        query_params = parse_qs(parsed_path.query)

        # Custom response content
        response = {
            "message": "GET request received",
            "path": parsed_path.path,
```

perfect! now instead of showing the response in an alert, show it in a new page. dont change anything else

Great! To show the interview questions on a new page instead of an alert, you'll need:

1. A new route in your React app (e.g. `/interview-questions`).
2. A component to display the questions.
3. To use `navigate()` in `handleGetInterviewQuestions` to go to that new page, passing the response via `state`.

✓ Step-by-step changes:

1. Create a new page component: `InterviewQuestions.js`

jsx

📄 Copiază

✎ Editează

```
import React from 'react';
import { useLocation, useNavigate } from 'react-router-dom';
import '../design/Application.css';

const InterviewQuestions = () => {
```

```
127.0.0.1 - - [05/May/2025 00:19:47] "OPTIONS /suggestionsCV?
path=/Documents/Resumes/51ea4abd-7fa7-4fbd-ab26-
bfc6d0d98d23.pdf HTTP/1.1" 200 -
127.0.0.1 - - [05/May/2025 00:21:32] "OPTIONS /suggestionsCV?
path=/Documents/Resumes/51ea4abd-7fa7-4fbd-ab26-
bfc6d0d98d23.pdf HTTP/1.1" 200 - Is this working?
```

The log entries you provided indicate that an `OPTIONS` HTTP request was made to the endpoint `/suggestionsCV` with a `path` query parameter pointing to a specific PDF file.

The status code `200` means the server is successfully responding to the `OPTIONS` request, which is a type of HTTP request commonly used in Cross-Origin Resource Sharing (CORS) pre-flight checks. This typically occurs when the browser is making a request to a different origin, such as calling an API from a different domain or port.

However, an `OPTIONS` request by itself doesn't mean the actual data has been returned. It's simply a CORS pre-flight request to check if the server is willing to accept the real request (like a `GET` or `POST`).

To determine if it's working:

- 1. Check for the CORS response headers:** Ensure that the server is responding with the proper CORS headers to allow the client to make the actual request (like `Access-Control-Allow-Origin`, `Access-Control-Allow-Methods`, etc.).
- 2. Actual Data Request:** After the `OPTIONS` request, the client will usually send the actual `GET` or `POST` request (not just `OPTIONS`) to fetch the data. Check if you see any `GET` or `POST` requests following the `OPTIONS` request, and check if they are successful (status code `200` or other valid status codes like `201`).



